

AFRIBN — Developer Onboarding & System Overview

Welcome to AFRIBN. This document is the starting point for any engineer joining the team. It explains what the platform does, how it is built, how to run it, and where it is headed.

Audience: new backend/frontend engineers. Assumes comfort with JavaScript, HTTP, and Git. No framework experience required — the stack is deliberately minimal.

1. What AFRIBN is

AFRIBN is a strategic news-intelligence platform for Africa. It turns the flood of African news, policy, corporate and market information into **decision-grade intelligence** for investors, institutions, governments, diplomats and risk teams operating on the continent.

The product is not a news reader. It runs a full **intelligence lifecycle**: discover sources → collect → distil with AI → score for strategic value → verify → publish — and surfaces the result through a feed, country briefs, dashboards, alerts and reports.

Launch scope: 10 strategic markets

Coverage is deliberately focused on ten priority markets:

****Nigeria, South Africa, Kenya, Egypt, Morocco, Ghana, Rwanda, Côte d'Ivoire, Tanzania, Uganda.****

These are defined once in `src/strategic-markets.js` and drive country pages, filters, relevance weighting and the market selector everywhere in the UI.

The strategic lens

Intelligence is organised by a **Tier 1 / Tier 2 taxonomy** of what actually moves decisions in Africa (see `src/topics.js` and `src/relevance.js`):

- **Tier 1 — moves capital & operations:** political stability, security &

conflict, sovereign & macro risk, energy & resources, sanctions & expropriation.

- **Tier 2 — reshapes the environment:** policy & regulation, geopolitics, infrastructure, trade & investment, governance signals.

This taxonomy is the backbone of relevance scoring and the Topics product.

2. The intelligence lifecycle (the core domain)

This is the heart of the system. Every object is a stage in a pipeline

(src/pipeline.js):

```
Source
→ ScrapeJob          (a collection run)
→ RawArticle         (collected, deduped content)
→ Story / Event      (AI-distilled structured intelligence item)
→ Score              (signal / impact / confidence / risk ... )
→ [GapReport → FieldTask → AgentReport] (optional field-verification chain)
→ VerificationReview (editorial review)
→ PublishedIntelligence
→ Feed / Country Briefs / Dashboards / Alerts / Reports
```

Two important design decisions to understand early:

1. **The editorial state machine owns workflow status.** A story moves `draft → submitted_for_verification → verified → approved → published` via `transitionIntelligenceItem`. This is the canonical path and the publish gate. The heavier field-evidence chain (`gap → field task → agent report`) is an **optional enrichment** that raises confidence; it does **not** change a story's status.

2. **Scoring is wired into the pipeline.** Stories are auto-scored on distillation and re-scored on approval, so the feed, dashboards and alert rules always have real signal/confidence numbers (`autoScoreStory`).

There is **one** publish implementation (`publishStory`) and **one** scoring engine (`calculateAndStoreScore` in `src/scoring-service.js`) — earlier duplicates were consolidated. Don't reintroduce parallel paths.

3. Technology & engineering

Guiding principle: minimal dependencies

The backend uses **zero npm runtime dependencies** — only Node.js built-ins (`node:http`, `node:crypto`, `node:sqlite`, `global fetch`, ...). The frontend is **vanilla HTML/CSS/JS** with no framework or build step. This keeps the system easy to reason about, fast to deploy, and free of supply-chain churn. Preserve this unless there is a strong reason not to.

Stack at a glance

Layer	Choice
Runtime	Node.js ≥ 20 , CommonJS
HTTP server	<code>node:http</code> (no Express) — <code>server.js</code>
Routing	Custom router — <code>src/router.js</code>
Storage	SQLite via <code>node:sqlite</code> , document-style — <code>src/store.js</code> (Postgres is the migration target; see <code>migrations/</code> + <code>scripts/migrate-postgres.mjs</code>)
Frontend	Static pages in <code>AFRIBN_design/</code> , shared runtime <code>assets/afribn.js</code> , API client + page logic <code>assets/backend.js</code>
AI	OpenAI Responses API (strict JSON schema) + Qwen/DashScope, with a rules-based fallback — <code>src/openai-client.js</code>
Auth	JWT (HS256) — <code>src/auth.js</code> ; Google OAuth — <code>src/oauth.js</code>
Payments	Stripe via REST (no SDK) — <code>src/billing.js</code>
Deploy	Docker → Render, auto-deploy on push to GitHub <code>main</code>

Key subsystems

- **Collection engine** — `src/scrapers.js`. Multi-strategy discovery:

API **adapters** (`src/adapters/`: GDELT, NewsAPI, NewsData.io, World News),

RSS/Atom parsing, and **HTML-index** parsing with article-link scoring.

Crawls via `fetch` or **Playwright** (auto-falls back to `fetch`). Extraction is

a "readability-lite" pass. Includes dedupe (URL/checksum/title), and

conditional fetch (ETag / If-Modified-Since + body content hashing) so

unchanged pages are skipped.

- **Relevance / quality gate** — `src/relevance.js`. Cheap, no-LLM scoring of an

item against a **source mandate** (`{countries, sectors, includeKeywords,`

excludeKeywords}`). The 10 launch markets get top geo weight; Tier 1 signals outrank Tier 2. This is what stops the pipeline ingesting noise and what keeps AI spend focused on relevant items.

- **AI distillation** — `src/openai-client.js`. Converts a raw article into a structured intelligence item (title, summary, key facts, why-it-matters, consequences, what-to-watch, entities, impact, value). Strict JSON schema on OpenAI; provider auto-selected (Qwen if keyed, else OpenAI, else rules fallback) so it always returns a valid object.

- **Scoring** — `src/scoring.js` + `src/scoring-service.js`. Eight score types (impact, signal, confidence, risk, decision-relevance, negotiation-leverage, scenario, intelligence-value). Factors are **auto-derived** from the story/event/sources/context, so scoring needs no manual input.

- **Source-reliability engine** — `src/source-reliability*.js`. Source Reliability Score, collection-priority decisions, blacklist/whitelist, and scrape-depth gating (metadata / sample / partial / full).

- **Scheduler (automation)** — `src/scheduler.js`. Tiered + adaptive cadence, per-domain rate limiting, conditional fetch; after collecting it runs triage (relevance filter → distil relevant items → score → promote). Driven by the worker queue (`src/workers/`). **Off by default** (`SCHEDULER_ENABLED`).

- **Auth & RBAC** — `src/auth.js`, `src/middleware/rbac.js`. Stateless JWTs; every protected endpoint calls `requirePermission`. Roles: admin, analyst, agent, verifier, client. The session role is authoritative from the server (not client-switchable).

- **Billing** — `src/billing.js`. B2B, per-organisation, seat-based subscriptions (Analyst / Professional / Enterprise) with trial → active → past_due → canceled lifecycle, plan **entitlements**, and Stripe Checkout + webhooks. Degrades gracefully to a trial when Stripe keys aren't set.

- **Products**. Feed, country briefs (`src/products.js` + `country.html`), Topics (`src/topics.js`), policy monitor, deal tracker, event tracker, risk dashboard, reports (PDF via `src/pdf.js`), watchlists, alerts

(src/alert-delivery.js: SendGrid email / Twilio SMS), search (src/search/), and a lightweight knowledge graph (src/graph/).

Request & URL model

- Pages are served at **clean URLs** (/, /pricing, /feed, /country?country=Kenya). The JSON **API lives under** /api/*. Legacy /design/*.html and *.html paths **301-redirect** to canonical URLs. Static assets are at /assets/*. All of this is handled in server.js.
- The frontend talks to the API through a single client (backend.js
API.base = "/api"); pages add behaviour through page-keyed init handlers.

Security posture

- Server-side **auth gate** on all data endpoints (public allowlist is tiny).
- **CSP + HSTS** headers; **CORS** restricted to an allowlist (ALLOWED_ORIGINS).
- All secrets via environment variables; .env is git-ignored and never shipped to the client. Integrations fail safe (AI → rules, Stripe → trial, OAuth button → disabled) when unconfigured.

4. Repository layout

```
server.js           # HTTP server: headers, CORS, /api routing, static + redirects, scheduler boot
Dockerfile          # production image (Render)
src/
  router.js         # all API routes
  store.js          # SQLite document store
  pipeline.js       # the intelligence lifecycle
  scrapers.js       # collection engine
  relevance.js      # mandate + Tier 1/2 relevance scoring
  openai-client.js  # AI distillation (OpenAI / Qwen / fallback)
  scoring.js / scoring-service.js
  source-reliability*.js
  scheduler.js      # automation loop
  seed-sources.js   # seeds 1 source per market to start automation
  billing.js        # Stripe subscriptions
  topics.js         # Tier 1/2 topic catalog
  strategic-markets.js # the 10 launch markets
  auth.js / oauth.js / middleware/rbac.js
  adapters/         # GDELT, NewsAPI, NewsData, World News
  workers/          # job queue + processor
  graph/ search/ products.js pdf.js alert-delivery.js uploads.js
  config.js env.js http.js rate-limit.js seed.js
AFRIBN_design/      # frontend (static pages + assets/)
tests/              # plain-node assertion scripts (no framework)
scripts/            # one-off maintenance scripts
docs/               # this file + design/TDR docs + ADRs
migrations/         # Postgres migration target
```

5. Running it locally

```
cp .env.example .env      # fill in keys as needed (all optional to boot)
npm start                  # http://127.0.0.1:3000
npm test                   # runs all test suites
```

- App: `http://127.0.0.1:3000/` (marketing) and `/login` → app pages.
- Local data lives in `data/afribn.sqlite` (git-ignored). Delete it to reseed.
- **Demo accounts** are seeded (e.g. `admin@afribn.local` / `afribn-admin-demo`,

plus `analyst/agent/verifier/client`). ****Rotate/remove these before production.****

- AI works offline via the rules fallback; set `OPENAI_API_KEY` or `QWEN_API_KEY` for real distillation.

Testing

There is **no test framework** — tests are plain Node scripts that `assert` and exit non-zero on failure (`tests/*.mjs`, run by `npm test`). Add a focused script per feature and wire it into the `test` script in `package.json`.

Deployment

`main` is the production branch. Pushing to GitHub

(github.com/zackhillmedia/afribn) triggers a **Render** redeploy (Docker).

Secrets are configured as Render environment variables, not in the repo. Always run `npm test` before pushing.

6. Turning on automation

The collection loop is built but **dormant by default**. To run it unattended:

1. **Seed sources:** `POST /api/sources/seed-markets` (admin) — adds one feed per market with its mandate.

2. **Enable on Render:** `SCHEDULER_ENABLED=true` (+ `SCHEDULER_TICK_MS`, `SCHEDULER_TRIAGE=true`).

3. **Keep the instance always-on** (the scheduler is an in-process timer).

It then runs: `scrape` → `trriage` (relevance) → `AI distil` → `score` → `promote a draft`.

The final **verify** → **publish** step is human (editorial control is intentional).

7. Conventions & gotchas

- **No npm runtime deps, no frontend build step.** Keep it that way.
 - **One path per concern** — single publish path, single scoring engine; don't fork them.
 - **Env-config everything**, and degrade gracefully when a key is missing.
 - The **role/session is server-authoritative**; never trust client-side role.
 - For **RSS sources**, set `scrapingConfig.crawlArticles = false` (crawling Google News redirect links overwrites titles with "Google News").
 - Design decisions are recorded as **ADRs** in `docs/` (see `ADR-001-collection-scheduler-and-resilience.md`). Add one for significant changes.
-

8. Where it's headed (future versions)

Near term

- **Scheduler Phase 3 — resilience:** proxy pool, tiered fetch fallback chain (RSS → proxy → headless → aggregator), block detection and a circuit breaker. This unblocks sources that 403 (e.g. igihe.com) or are JS-only (e.g. NewsNow) and makes collection robust at scale. (Design in ADR-001.)
- **Source breadth:** real per-country publisher feeds on distinct domains (today's seeder is all Google News, so the per-domain limit serialises it).
- **Cross-source clustering / corroboration:** collapse the same event from many outlets into one item with a corroboration count (itself a signal).
- **Entitlement enforcement everywhere:** plan entitlements are surfaced but not yet enforced on every endpoint.

Platform

- **Move the scheduler to a dedicated worker** (out of the web process) and **Postgres** for scale (migration path already stubbed).
- **Observability:** structured logging, metrics, job/queue dashboards.
- **Multilingual** distillation (French, Arabic, Portuguese, Swahili — the

Language field already flows through).

- **Knowledge graph & entity profiles** (companies, people, projects) as a first-class product.

Product

- Optional **auto-publish** for high-confidence items (behind strict signal + confidence thresholds) for a faster "real-time" tier.
- **Billing depth:** self-serve account management UI, and African payment rails (Paystack / Flutterwave) alongside Stripe.
- **More markets** beyond the launch ten; richer country and sector products.
- **API productisation** and a mobile client.

9. Quick reference

Task	Where
Add an API route	src/router.js
Change the lifecycle	src/pipeline.js
Tune relevance / topics	src/relevance.js, src/topics.js
Add a collection adapter	src/adapters/ + src/adapters/index.js
Adjust scoring	src/scoring.js, src/scoring-service.js
Markets / countries	src/strategic-markets.js
Plans / billing	src/billing.js
A page's behaviour	AFRIBN_design/assets/backend.js (page init handlers)
Shared UI (nav, flags, icons)	AFRIBN_design/assets/afribn.js
Env/config flags	.env.example, src/config.js

Welcome aboard — start by running it locally, signing in as `admin`, and walking a story from the feed through the workspace and verification to see the lifecycle end to end.